APPENDIX

In this appendix, we present a rigorous mathematical framework summarising the results in this paper. We also detail further algorithms that may be useful in the implementation of Phylo2Vec.

Notations and definitions

Notations We shall use the notation $\mathcal{T}$ to refer to a tree, $\boldsymbol{b}$ for the branch lengths of $\mathcal{T}$ and n as the number of leaves (or taxa).

Node labels In a tree with n leaves, we set the convention that the leaf nodes are *labelled* from 0 to $n-1$, the internal nodes are *labelled* from n to $2n-3$ and the root (if it exists) is labelled as $2n-2$.

Generation By definition, there is a unique path connecting each pair of nodes in a tree. Suppose that the path from node i to the root contains the nodes $i, a_1, \ldots, a_{g_i}$ in that order (and the root is hence a_{g_i}). We call g_i the *generation* of node i. The path from any a_x to the root must be $a_x, a_{x+1}, \ldots, a_{g_i}$ and hence $g_{a_x} = g_i - x$.

Unrooting Suppose that, in a rooted tree, the two children of the root are x and y. To "unroot" the tree, we remove the edges connecting x and y to the root and add a new edge connecting x and y. This edge is given length $b_x + b_y$ where, for example, b_x is the length of the original edge joining x to the root. We shall use $\mathcal{T}'$ to refer to the unrooted tree and $\boldsymbol{b}'$ to its branch lengths.

Phylo2Vec details

Vector representation As described in the main text, Phylo2Vec is a way to represent binary trees with n leaves using a single integer vector, $\boldsymbol{v}$ of dimension $n-1$ that is simply constrained by

$$\boldsymbol{v}[j] \in \{0, 1, \ldots, 2(j-1)\} \; \forall j \in \{1, \ldots, n-1\}$$

The construction of the tree from this representation and the bijectivity between $\boldsymbol{v}$ and the space of trees are covered in the main text.

Bijectivity of $\boldsymbol{v}$ to the space of all possible trees We can show that our mapping from $\mathbb{V}$ to the space of trees is a bijection.

LEMMA A.1 *The mapping between the set of vectors $\boldsymbol{v} \in \mathbb{V}$ and the set of (topologically equivalent, labelled) trees is a bijection.*

Proof: As, by construction, the number of possible vectors $\boldsymbol{v}$ and the number of trees are the same ($(2n-3)!!$), it is simply necessary to show that this mapping is injective.

For any two nodes a and b, we define $M(a, b)$ to be their most recent common ancestor (MRCA). Moreover, for any nodes a and b, we use the notation $a \prec b$ if a is an ancestor of b.

We can use the fact that the MRCA of any pair of leaf nodes in the tree is unchanged during the Phylo2Vec construction process (once both nodes have been added to the tree). Note that the path from a node x to the root changes through the addition of an extra node if and only if a new node is appended to an edge on this path. Thus, if $M(a, b) = x$, x will remain on the paths from a and b to the root. Moreover, nodes of higher generation than x can only be added to one of the paths (as otherwise there would have to be an edge connecting at least one node of higher generation than x on both the original paths, contradicting x being the MRCA). Thus, the MRCA of a and b will be unchanged. Similarly, we can see that $a \prec b$ for two a and b at a given stage of the algorithm, this relationship will be unchanged throughout the construction process.

Suppose that two vectors $\boldsymbol{v}$ and $\boldsymbol{v}'$ result in trees $\mathcal{T}$ and $\mathcal{T}'$, and that $\boldsymbol{v} \neq \boldsymbol{v}'$. Define $i := \min\{j : v_j \neq v'_j\}$.

Define X to be the set of leaf nodes that, just before node i is added, are descended from the edge to which node i is appended when constructing the tree according to $\boldsymbol{v}$, and X' to be the equivalent set for $\boldsymbol{v}'$. We have $X \neq X'$, as each edge has a unique set of nodes descended from it.

If both $X \setminus X'$ and $X' \setminus X$ are non-empty, suppose that $a \in X \setminus X'$ and $b \in X' \setminus X$. Then, in $\mathcal{T}$, it must be the case that $M(b, i) \prec M(a, i)$ (both at this stage in the algorithm, and in

the final tree, by the previous argument) once node i has been added (as the $M(a,i)$ will be the new internal node and $M(i,b)$ cannot be either i or this new internal node). A similar argument shows that $M(a,i) \prec M(b,i)$ in $\mathcal{T}'$ and hence $\mathcal{T} \neq \mathcal{T}'$.

If only one of $X \setminus X'$ and $X' \setminus X$ is non-empty, suppose without loss of generality that $X \setminus X'$ is non-empty and choose $a \in X \setminus X'$. We can choose a distinct node $b \in X' \cap X$ (as otherwise, if $X' \cap X$ and $X' \setminus X$ are both empty, we must have one of X and X' being empty which is a contradiction as every edge has at least one leaf node descended from it). Then, $M(i,a)$ is the newly-added internal node in the construction of $\mathcal{T}$, but not in the construction of $\mathcal{T}'$. However, in both cases, $M(i,b)$ is the newly-added node. Hence, in $\mathcal{T}$, $M(i,a) = M(i,b)$, but in $\mathcal{T}'$, they are distinct. Thus, $\mathcal{T}' \neq \mathcal{T}$.

Hence, topological non-equivalence holds in both cases, and the map from $\boldsymbol{v}$ to the set of trees is therefore injective and therefore bijective.

Label-asymmetry of $\boldsymbol{v}$-induced distance As discussed in the main text, $\boldsymbol{v}$ induces a natural distance function between trees - namely, that the distance between $\boldsymbol{v}$ and $\boldsymbol{w}$ is equal to $\sum_{i=0}^{k} \mathbb{I}_{v_i \neq w_i}$.

However, this distance function is dependent on the labels assigned to each leaf (and is hence label-asymmetric). A simple example of this can be found in the case of four leaves.

Consider the tree given by $\boldsymbol{v}_1 = (0,1,2)$. This is a ladder tree (that is, each internal node and the root is parent to at least one leaf node) with nodes in order $0, 1, \{2,3\}$ (where the $\{2,3\}$ is used to denote the fact that nodes 2 and 3 are from the same generation and so could be read in either order). This is distance 1 away from $\boldsymbol{v}_2 = (0,1,4)$, which is again a ladder tree with nodes now ordered as $3, 0, \{1,2\}$. Thus, if distance were symmetric, then any ladder tree with ordered nodes $a, b, \{c,d\}$ would be distance 1 away from the ladder trees with ordered nodes $d, a, \{b,c\}$ and $c, a, \{b,d\}$. However, the ladder tree with ordered nodes $2, 0, \{1,3\}$ is given by $\boldsymbol{v}_3 = (0,2,1)$, which is distance 2 away from $\boldsymbol{v}_1$. Thus, the distance function is not label-symmetric.

Unrooted tree equivalence classes Noting from the main text that there are $(2k-3)!! = \prod_{i=0}^{k-2}(2i+1)$ unrooted trees with $k+1$ leaves, we can partition the space of trees

with $k+1$ leaves into $(2k-3)!!$ equivalence classes $\{\mathcal{E}_i : i = 1, 2, \ldots, (2k-3)!!\}$ such that the removing the root from each of the trees in a given class $\mathcal{E}_i$ (according to the procedure described in Notations and definitions in the Appendix) results in the same unrooted tree. These equivalence classes all contain $(2k-1)$ trees (as, reversing the unrooting procedure, a root can be added onto each of the $(2k-1)$ edges of an unrooted tree).

From the previous work, Felsenstein's likelihood is constant in each equivalence class. This has two consequences when attempting to maximise the likelihood. Firstly, a global minimum in the space of rooted trees will not be unique, as any of the other trees in the same equivalence class will also give a maximal likelihood. Secondly, if $\boldsymbol{v}$ is a local maximum (that is, all $\boldsymbol{w}$ with one entry different to $\boldsymbol{v}$ give a lower likelihood value), equivalence classes provide a way to change to a new vector $\boldsymbol{u}$ without having to decrease the likelihood.

Similarly to the case of reordering, the set of equivalence classes of vectors that are distance 1 from $\boldsymbol{v}$ will not in general be the same as the set with distance 1 from $\boldsymbol{u}$. One can either change equivalence class randomly or (unlike in the case of reordering), as there are only $2k-1$ members of each class, systematically iterate through them. Finding the vectors $\boldsymbol{w}$ corresponding to the trees in an equivalence class can again be done by constructing the tree matrix M, making the appropriate changes and then using the inversion algorithm to create a new vector representation.

Our experiments have shown that changing $\boldsymbol{v}$ is effective in increasing algorithmic performance. However, we found that reordering provided better increase in performance, and therefore did not present this other switching method in the algorithm in the main text.

Number of moves Let a vector $\boldsymbol{v}$ of length $n-1$ representing a tree with n leaves. For each entry j, there are $2j-1-1 = 2(j-1)$ possible moves, as there are $2j-1$ possible entries for entry j, but $\boldsymbol{v}[j]$ is already set). Summing for all $j \in 1, \ldots, n-1$ gives:

$$\sum_{j=1}^{n-1} 2(j-1) = (n-1)(n-2) \approx n^2$$

Thus, the number of possible moves for Phylo2Vec is of the order $\mathcal{O}(n^2)$.

Algorithms

The algorithms provided here are presented in rough descriptive pseudocode; they are written in human-readable fashion to facilitate comprehension, but are not necessarily optimised. Please refer to the GitHub repository (https://github.com/Neclow/phylo2vec) for detailed implementation.

Algorithm S1 Labelling a rooted tree as an ordered $\boldsymbol{v}$

Input rooted tree $\mathcal{T}$ with n leaves

$\boldsymbol{v} \leftarrow [0]$ ▷ Initial integer vector

for all leaves $j = 2, ..., n-1$ **do**

  $\mathcal{T}_j \leftarrow$ subtree with leaves $0, ..., j$

  $s \leftarrow$ leaf in $\mathcal{T}_j$ such that j and s form a cherry

  $\boldsymbol{v}$.append(s)

end for

return $\boldsymbol{v}$

Algorithm S2 Recovering an ordered rooted tree from an ordered $\boldsymbol{v}$

Input $\boldsymbol{v}$ of size $n-1$ satisfying Eq. 1 ▷ Ordered integer vector

$\mathcal{T} \leftarrow$ a rooted tree with two leaves 0 and 1 ▷ Initial tree

for all $j = 2, ..., n-1$ **do**

  Add a new leaf j to $\mathcal{T}$ such that it forms a cherry with $\boldsymbol{v}[j]$

  $a(j, \boldsymbol{v}[j]) \leftarrow 2(n-1) - j + 1$ ▷ Ancestor of j and $\boldsymbol{v}[j]$

end for

return $\mathcal{T}$

Algorithm S3 Recovering a rooted tree from a Phylo2Vec $\boldsymbol{v}$

Input $\boldsymbol{v}$ of dimension $n-1$ satisfying Eq. 2 ▷ Phylo2Vec vector

pairs $\leftarrow [(0,1)]$ ▷ The objective of the algorithm is find the nodes to merge and their order. The first pair is always (0,1) for a 2-leaf tree.

for all leaves $j = 2, ..., n-1$ **do**

  if $\boldsymbol{v}[j] \leqslant j$ **then** ▷ This is like an ordered vector. The branch leading to $\boldsymbol{v}$[j] gives birth birth to leaf j

    next pair $\leftarrow (\boldsymbol{v}[j], j)$

    position $\leftarrow$ 1 ▷ This pair corresponds to a cherry at height 0, so it should be drawn first.

  else ▷ The branch to be split is internal

    position $\leftarrow \boldsymbol{v}[j]$ - len(pairs) + 1 ▷ The index at which the next pair will be inserted

    descendant $\leftarrow$ pairs[position - 1][1] ▷ A node that we processed beforehand that is deeper than the branch $\boldsymbol{v}[j]$

    next pair $\leftarrow$ (pairs[descendant, j])

  end if

  pairs.insert(position, next pair)

end for

return pairs ▷ The pairs indicate how to build the tree (and form a Newick string)

Algorithm S4 Labelling a rooted tree as a Phylo2Vec vector

Input rooted tree r with n leaves ▷ We assume a Newick string with integer nodes and labelled internal nodes.
Ex: (((2,3)6,1)7,(0,4)5)8;

$M \leftarrow$ reduce(r) ▷ "Reduce" the Newick string to a $(n-1) \times 3$ matrix.
Columns 1-2 = children nodes
Column 3 = ancestor
Ex:
$$\begin{matrix} 0 & 4 & 5 \\ 2 & 3 & 6 \\ 1 & 6 & 7 \\ 5 & 7 & 8 \end{matrix}$$

$C \leftarrow$ extract_cherries(M) ▷ Starting from the smallest internal node, replace the internal nodes by their smallest child (and discard the third column). This is equivalent to the pairs in Algorithm S3
Ex:
$$\begin{matrix} 0 & 4 \\ 2 & 3 \\ 1 & 2 \\ 0 & 1 \end{matrix}$$

$\boldsymbol{v} \leftarrow$ build_vector(C) ▷ Use the same logic as Algorithm S3 to retrieve each $\boldsymbol{v}_j$ from the position of the containing leaf j

return $\boldsymbol{v}$

Supplementary Figures

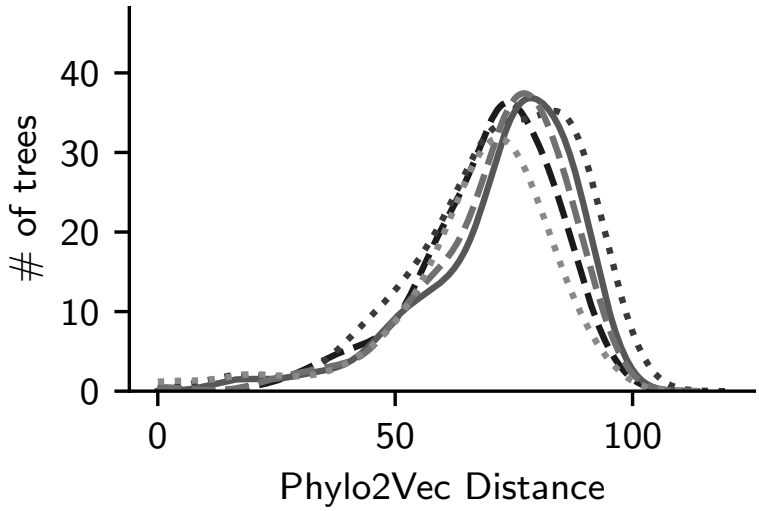

Fig. S1. Density plots of Phylo2Vec distance μ for an equivalence set of rooted trees with 200 taxa with a fixed Phylo2Vec vector $\boldsymbol{v}$. 5 random $\boldsymbol{v}$ are shown.

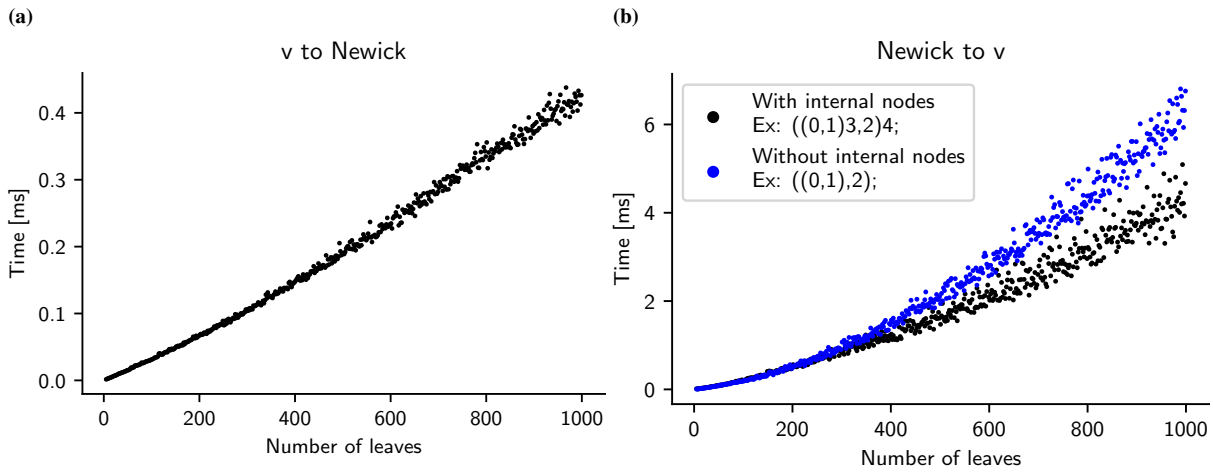

Fig. S2. Execution times for different tree sizes of **(a)** Algorithm S3 to recover a rooted tree from a Phylo2Vec $\boldsymbol{v}$ and **(b)** Algorithm S4 to label a rooted tree as a Phylo2Vec $\boldsymbol{v}$. For each size, we evaluated the execution time with a fixed configuration of 100 executions in a loop and 7 repeats using `timeit` in Python. When the number of taxa is large, manipulations on long strings to produce or process the Newick string can increase memory and thus increase execution time.